

NOT IN vs NOT EXISTS

#sql-server #correctness #null-handling #refactoring

The Short Version

NOT IN against a subquery is unsafe in the presence of NULL. If any single row in the subquery returns NULL for the compared column, the entire NOT IN predicate evaluates to UNKNOWN, which excludes every outer row from the result. The query silently returns zero rows, or fewer rows than expected, with no error and no warning. NOT EXISTS does not have this problem and produces the result the author almost always intended.

The fix is mechanical. Convert every NOT IN (Select Col From ...) to NOT EXISTS (Select 1 From ... Where Col = OuterCol). The logic is more explicit, the NULL handling is correct, and the optimizer is generally able to produce a better plan because the correlation is visible in the predicate rather than buried in a subquery materialization.

Why NOT IN Breaks on NULL

Standard SQL three-valued logic treats NULL as UNKNOWN. The expression X NOT IN (A, B, NULL) expands logically to X <> A AND X <> B AND X <> NULL. The third comparison is UNKNOWN no matter what X is, because nothing equals or is unequal to NULL. UNKNOWN combined with anything via AND produces UNKNOWN. The whole predicate is UNKNOWN. The row is filtered out.

Concretely:

```
-- StoreA values: ('100', '200', NULL)
-- StoreB values: ('100', '200', '300')

Select Distinct StoreNum
From StoreB
Where StoreNum Not In (Select StoreNum From StoreA);

-- Expected (intuitive reading): '300'
-- Actual: zero rows
```

The single NULL in StoreA's StoreNum column poisons the entire predicate. The query author almost certainly meant “give me StoreB stores that don't have a matching row in StoreA,” but that is not what NOT IN says when NULLs are present.

Why NOT EXISTS Is Safe

NOT EXISTS (Select 1 From StoreA Where StoreA.StoreNum = StoreB.StoreNum) reads as “no row in StoreA has a matching StoreNum to this StoreB row.” The correlation is explicit. NULL does not match NULL in the join predicate (because NULL = NULL is UNKNOWN, not true), so the NULL row in StoreA is not considered a match for any StoreB row. The query returns the rows the author expected.

```
Select Distinct StoreNum
From StoreB
Where Not Exists (
    Select 1 From StoreA Where StoreA.StoreNum = StoreB.StoreNum
);

-- Returns: '300'
```

This is the form to standardize on. Any time you see `NOT IN (Select ...)` against a column that could contain `NULL`, treat it as a correctness bug, not a performance issue.

When NOT IN Is Safe

`NOT IN` is safe in two narrow cases:

1. The subquery's column has a `NOT NULL` constraint. The schema guarantees no `NULL` can ever appear, so the predicate cannot be poisoned. This is the only ironclad case.
2. The subquery has an explicit `WHERE Col Is Not Null` filter. The filter strips any `NULL`s before the predicate evaluates.

In every other case, `NOT IN` is a latent bug waiting for a `NULL` to slip into the data. Even if the column today has no `NULL`s in production, a future schema change, a bad import, or a one-off manual update can introduce one. Once the `NULL` appears, the `NOT IN` predicate silently zeroes out the result set, and the bug is invisible until someone notices the missing rows.

`NOT IN` against a literal list of values is safe so long as no literal in the list is the `NULL` keyword. Where `StoreNum Not In ('100', '200', '300')` is fine. Where `StoreNum Not In ('100', NULL, '300')` is not.

Performance, as a Side Benefit

The optimizer often produces a better plan for `NOT EXISTS` than for `NOT IN`. The `NOT IN` form forces the engine to materialize or repeatedly scan the subquery's result set and compare each outer row against the full set. The `NOT EXISTS` form is naturally a left anti-semi join, which is a well-understood operator with good cost estimates and short-circuit evaluation. The engine stops looking as soon as it finds the first matching row in the subquery, which can be dramatically cheaper than the materialize-and-compare approach.

In practice, the plan-shape difference is most visible when the subquery returns many rows. For small subqueries the two forms compile to nearly identical plans. For large subqueries, `NOT EXISTS` is reliably faster, sometimes by an order of magnitude.

How to Spot the Pattern

Search the proc body for the literal string `Not In` or `NOT IN` (case-insensitive). For each hit, ask three questions in order:

1. Is the right-hand side a subquery, or a literal list? Literal lists are usually fine. Subqueries need scrutiny.
2. Could the column referenced inside the subquery's `SELECT` list be `NULL`? Check the table DDL. Any column without `NOT NULL` is suspect.
3. Does the subquery already filter `NULL`s out via `WHERE Col Is Not Null`? If yes, the predicate is safe; if no, it is broken.

If the answer to question 2 is yes and question 3 is no, the predicate has a `NULL`-safety bug. Convert to `NOT EXISTS`.

The Mechanical Conversion

```
-- Before
Where OuterCol Not In
(
  Select InnerCol
  From SomeTable
  Where <inner conditions>
```

```

)
-- After
Where Not Exists
(
    Select 1
    From SomeTable
    Where InnerCol = OuterCol
    And <inner conditions>
)

```

Two specific things to watch:

- The inner SELECT changes from `Select InnerCol` to `Select 1`. The actual returned column does not matter for EXISTS; the engine only cares about row presence. `Select 1` is the canonical form.
- The correlation predicate `InnerCol = OuterCol` moves into the inner WHERE. This is the explicit join condition that makes the semantics clear. If the original subquery already had a WHERE clause, AND the correlation onto the existing one.

Related Concepts

- **Correlated Subqueries to CTEs**: when the inner subquery is expensive and reused across multiple call sites, materialize once into a temp table and probe via EXISTS.
- **LEFT JOIN OR Anti-Pattern**: another pattern where rewriting to EXISTS against a materialized set is the common fix.
- **Non-SARGable Predicates**: `NOT IN` and `NOT EXISTS` are both subject to the same SARGability rules on the columns they reference. The predicate form does not rescue a non-SARGable column reference.